

day 43 middlewares

Express.js Middleware — Complete Notes

Day 43 | Apna College Web Dev

1. What is Middleware?

Definition: Middleware is an **intermediary** — a function that sits between the **request** and the **response**.

text

```
Client → Request → [ Middleware ] → Response → Client
```

In Express: Middleware functions run **after the server receives the request** and **before the response is sent** to the client.

Real-life analogy: Think of a security guard at a building entrance. Before you reach your destination (route handler), you pass through the guard (middleware) who checks your ID, logs your entry, etc.

2. What Can Middleware Do?

Task	Example
✓ Execute any code	Logging, validation
✓ Modify <code>req</code> and <code>res</code> objects	<code>req.user = user</code> , <code>req.time = Date.now()</code>
✓ End the request-response cycle	<code>res.send("bye")</code>
✓ Call the next middleware	<code>next()</code>

Golden Rule: Every middleware must either:

- **Send a response** (`res.send` , `res.json` , etc.) → cycle ends
- **Call** `next()` → pass control forward

⚠ If it does neither → the request **hangs** forever.

3. Types of Middleware

Type	How it's used	Scope
Application-level	<code>app.use(fn)</code>	Entire app
Route-level	<code>app.get("/path", fn, handler)</code>	Specific route only
Router-level	<code>router.use(fn)</code>	Specific Router instance
Built-in	<code>express.static()</code> , <code>express.urlencoded()</code>	Built into Express
Third-party	<code>body-parser</code> , <code>morgan</code> , <code>cors</code>	Installed via npm
Error-handling	<code>app.use((err, req, res, next) => {})</code>	Catches errors (4 args)

4. Built-in Middleware

`body-parser`

Parses data inside the **request body** (JSON, form data) and converts it into a readable JS object so you can use `req.body` .

js

```
app.use(express.json());           // for JSON data
app.use(express.urlencoded({ extended: true })); // for form data
```

`express.static`

Serves **static files** (CSS, JS, images) from the backend to the frontend.

js

```
app.use(express.static("public"));
```

express.urlencoded

Makes form data accessible via `req.body`.

5. Middleware Syntax & Chaining

Basic Syntax

js

```
app.use((req, res, next) => {
  console.log("I am middleware");
  next(); // move to next
});
```

Chaining (multiple middleware)

js

```
app.use((req, res, next) => {
  console.log("1st middleware");
  next();
});

app.use((req, res, next) => {
  console.log("2nd middleware");
  next();
});

app.get("/", (req, res) => {
  res.send("Hi, I am root.");
});
```

Order of execution for ANY request:

text

```
1st middleware → 2nd middleware → route handler
```

 **Critical:** Middleware runs in the **order it's defined**. Even for wrong/nonexistent routes, global middleware still runs.

6. `next()` — Deep Dive

js

```
app.use((req, res, next) => {
  console.log("Before next");
  next();
  console.log("After next"); // ⚠ THIS STILL RUNS!
});
```

Important: `next()` does **not** return — it just signals Express to continue. Code after `next()` will execute unless you `return`.

Best Practice: Use `return next()`

js

```
app.use((req, res, next) => {
  console.log("Before");
  return next(); // ✅ stops here
  console.log("Never runs");
});
```

Rule of thumb:

- If sending response → `return res.send(...)`
- If calling next → `return next()`

7. Middleware as an Array (Reusability)

js

```
function logUrl(req, res, next) {
  console.log("URL:", req.originalUrl);
  next();
}

function logMethod(req, res, next) {
  console.log("Method:", req.method);
}
```

```

    next();
  }

  const logStuff = [logUrl, logMethod];

  app.get("/user/:id", logStuff, (req, res) => {
    res.send("User Info");
  });

```

Why? Keeps code DRY — reuse the same middleware stack across routes.

8. Router-Level Middleware

Works the same as app-level, but attached to an `express.Router()`.

js

```

const router = express.Router();

router.use((req, res, next) => {
  console.log("Router middleware");
  next();
});

router.get("/", (req, res) => res.send("Router Home"));

app.use("/admin", router); // mounts router at /admin

```

Use case: Modular routing — e.g., `/users`, `/products`, `/admin` each get their own router.

9. Utility Middleware (Logger Example)

A middleware that adds useful functionality, like logging every request.

js

```

app.use((req, res, next) => {
  req.time = new Date(Date.now()).toString();
  console.log(req.method, req.hostname, req.path, req.time);
  next();
});

```

Output example:

text

```
GET localhost /api 2024-01-15T10:30:00.000Z
```

What it logs:

- `req.method` → GET/POST/PUT/DELETE
- `req.hostname` → domain
- `req.path` → route path
- `req.time` → timestamp

10. Order Matters! (Route vs Middleware)

js

```
// Case 1: Middleware FIRST
app.use(logger);
app.get("/", handler);
// → logger runs BEFORE handler

// Case 2: Middleware AFTER route
app.get("/", handler);
app.use(logger);
// → handler runs FIRST (if it sends response, logger never runs)
```

Rule: Express executes functions in **top-to-bottom order**.

11. Authentication Middleware (Token Example)

The Goal

Protect `/api` route — only allow access if a valid token is passed as a query string.

The Code

js

```
const checkToken = (req, res, next) => {
  let { token } = req.query;

  if (token === "giveaccess") {
    return next(); // ✅ valid token → continue
  }

  res.status(403).send("ACCESS DENIED!"); // ❌ invalid
};

app.get("/api", checkToken, (req, res) => {
  res.send("data");
});
```

How it works

URL	Token	Result
<code>/api?token=giveaccess</code>	✅ valid	<code>next()</code> → "data"
<code>/api?token=wrong</code>	❌ invalid	"ACCESS DENIED!"
<code>/api</code>	❌ missing	"ACCESS DENIED!"

⚠️ **Common Bug:** Missing `return` before `next()`. Without it, `res.send("ACCESS DENIED!")` runs too → error "Cannot set headers after they are sent".

12. Error Handling

Throwing Errors

js

```
app.get("/wrong", (req, res) => {
  abcd = abcd; // ReferenceError!
});
```

Express Default Error Handler

When an error is thrown, Express catches it and sends:

- Status **500**

- Error message + stack trace (in development)

Custom Error Handler (4 arguments)

js

```
app.use((err, req, res, next) => {  
  console.error(err.message);  
  res.status(500).send("Something broke!");  
});
```

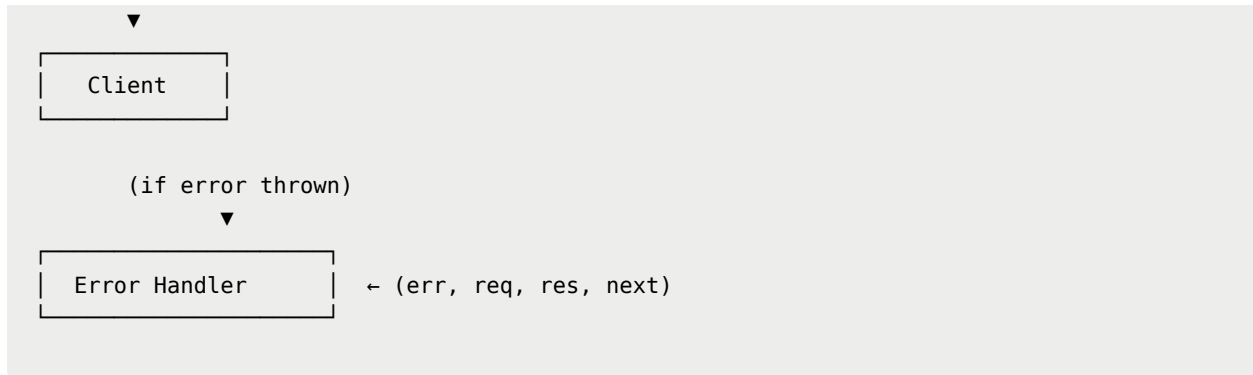
Key points:

- Must have **4 parameters**: `(err, req, res, next)`
- Must be defined **after all routes**
- Use `throw new Error("msg")` in middleware to trigger it

13. Complete Flow Diagram

text





14. Common Mistakes & Best Practices

❌ Mistake	✅ Fix
Forgetting <code>next()</code>	Always call <code>next()</code> or send response
Code after <code>next()</code> runs unexpectedly	Use <code>return next()</code>
Missing <code>return</code> before <code>res.send</code>	<code>return res.send()</code>
Error handler defined before routes	Place it after all routes
Error handler with 3 args	Must have 4 args <code>(err, req, res, next)</code>
Using <code>app.use()</code> for route-specific logic	Pass middleware directly to route: <code>app.get("/api", checkToken, handler)</code>

15. Quick Reference Cheatsheet

js

```

// — GLOBAL MIDDLEWARE —
app.use((req, res, next) => { /* ... */ next(); });

// — ROUTE MIDDLEWARE —
app.get("/path", middleware, (req, res) => { /* ... */ });

// — ARRAY MIDDLEWARE —
app.get("/path", [mw1, mw2], handler);

// — ROUTER MIDDLEWARE —
const router = express.Router();
router.use(mw);
app.use("/prefix", router);
  
```

```
// — ERROR HANDLER —
app.use((err, req, res, next) => {
  res.status(500).send(err.message);
});

// — BUILT-IN —
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));
```

16. Key Terms to Remember

Term	Meaning
Middleware	Function between request and response
<code>next()</code>	Pass control to next function
<code>req.query</code>	URL query params (<code>?token=x</code>)
<code>req.body</code>	Request body (needs body-parser)
<code>req.params</code>	Route params (<code>/user/:id</code>)
<code>req.method</code>	HTTP verb (GET/POST/etc.)
<code>app.use()</code>	Register middleware globally
Error handler	4-arg middleware for catching errors

Summary — 5 Things to Remember

1. **Middleware** = **middleman** between request & response.
2. **Always** call `next()` or send a response — never both.
3. **Order matters** — middleware runs top-to-bottom.
4. `return next()` prevents code after from running.
5. **Error handler** = 4 args, defined last.

working code:

Project Setup

1. Create a folder and initialize:

bash

```
mkdir express-middleware-demo
cd express-middleware-demo
npm init -y
```

2. Install Express:

bash

```
npm install express
```

3. Create a file named `app.js` and paste the code below.

Complete `app.js` Code

js

```
const express = require("express");
const app = express();
const PORT = 8080;

// =====
// 1. GLOBAL / APPLICATION-LEVEL MIDDLEWARE
// =====

// --- Logger Middleware (Utility Middleware) ---
// Runs for EVERY request, no matter the route.
app.use((req, res, next) => {
  const time = new Date(Date.now()).toString();
  console.log(`[${time}] ${req.method} ${req.hostname} ${req.path}`);
  next(); // Pass control to the next middleware
});

// --- Second Global Middleware (to show chaining) ---
```

```

app.use((req, res, next) => {
  console.log("    -> Passed through 2nd global middleware");
  next();
});

// =====
// 2. ROUTE-SPECIFIC MIDDLEWARE
// =====

// --- Token Checker Middleware ---
// This is NOT global. It is passed directly into specific routes.
const checkToken = (req, res, next) => {
  let { token } = req.query;

  // If token is correct, pass control to the route handler
  if (token === "giveaccess") {
    return next(); // 'return' stops the function here so the error below doesn't run
  }

  // If token is wrong, send a 403 Forbidden response
  res.status(403).send("ACCESS DENIED! Invalid or missing token.");
};

// =====
// 3. ROUTES
// =====

// --- Public Route (No token needed) ---
app.get("/", (req, res) => {
  res.send("Hi, I am the root route. Everyone can see this.");
});

// --- Random Route (No token needed) ---
app.get("/random", (req, res) => {
  res.send("This is a random page.");
});

// --- Protected Route (Token required) ---
// 'checkToken' runs first. If it calls next(), this handler runs.
app.get("/api", checkToken, (req, res) => {
  res.send("🔒 Success! Here is your secret data.");
});

// --- Route with Multiple Middleware in an Array (Reusability) ---
function logOriginalUrl(req, res, next) {
  console.log("    -> [Array Middleware] Request URL:", req.originalUrl);
  next();
}

function logMethod(req, res, next) {
  console.log("    -> [Array Middleware] Request Type:", req.method);
  next();
}

```

```

}

const logStuff = [logOriginalUrl, logMethod];

app.get("/user/:id", logStuff, (req, res) => {
  res.send(`User Info for ID:${req.params.id}`);
});

// =====
// 4. ERROR HANDLING
// =====

// --- Route that intentionally throws an error ---
app.get("/wrong", (req, res) => {
  // This will throw a ReferenceError because 'abcd' is not defined.
  abcd = abcd;
});

// --- Custom Error-Handling Middleware ---
// Express recognizes this as an error handler because it has 4 arguments: (err, req, res, next)
// It MUST be defined AFTER all other app.use() and routes.
app.use((err, req, res, next) => {
  console.error("❌ Error Caught:", err.message);
  res.status(500).send("Something broke! Please try again later.");
});

// =====
// 5. START SERVER
// =====
app.listen(PORT, () => {
  console.log(`Server listening on port${PORT}`);
});

```

How to Test It (Step-by-Step)

Run the server:

bash

```
node app.js
```

Now, open your browser or use Postman to test these URLs. **Watch your terminal** to see the middleware logs in real-time.

Test 1: The Root Route (Public)

- **URL:** `http://localhost:8080/`
- **Result:** "Hi, I am the root route..."
- **Terminal Output:**

text

```
[Time] GET localhost/  
-> Passed through 2nd global middleware
```

Test 2: The Random Route (Public)

- **URL:** `http://localhost:8080/random`
- **Result:** "This is a random page."
- **Terminal Output:**

text

```
[Time] GET localhost/random  
-> Passed through 2nd global middleware
```

Test 3: Protected Route (No Token)

- **URL:** `http://localhost:8080/api`
- **Result:** "ACCESS DENIED! Invalid or missing token." (Status 403)
- **Terminal Output:**

text

```
[Time] GET localhost/api  
-> Passed through 2nd global middleware
```

Test 4: Protected Route (With Token)

- **URL:** `http://localhost:8080/api?token=giveaccess`
- **Result:** "🔓 Success! Here is your secret data."
- **Terminal Output:**

text

```
[Time] GET localhost/api  
-> Passed through 2nd global middleware
```

Test 5: Array Middleware Route

- **URL:** `http://localhost:8080/user/42`
- **Result:** "User Info for ID: 42"
- **Terminal Output:**

text

```
[Time] GET localhost/user/42  
-> Passed through 2nd global middleware  
-> [Array Middleware] Request URL: /user/42  
-> [Array Middleware] Request Type: GET
```

Test 6: Error Handling Route

- **URL:** `http://localhost:8080/wrong`
- **Result:** "Something broke! Please try again later." (Status 500)
- **Terminal Output:**

text

```
[Time] GET localhost/wrong  
-> Passed through 2nd global middleware  
❌ Error Caught: abcd is not defined
```

🔑 Key Things to Notice in This Code

1. **Order of Middleware:** The Logger (`app.use`) is defined at the very top. That's why it runs for *every* request, even the broken `/wrong` route.
2. `return next()` : In the `checkToken` middleware, we used `return next()` . This ensures that if the token is valid, the code below it (sending the 403 error) does **not** accidentally run.
3. **Route-Specific vs Global:** `checkToken` is not called with `app.use()` . It is only called inside the `/api` route. This means `/` and `/random` don't need a token.
4. **Array of Middleware:** The `/user/:id` route shows how you can group middleware into an array (`logStuff`) and reuse them across different routes.
5. **Error Handler:** Notice the error handler has **4 parameters** (`err, req, res, next`). It must be placed *after* all your routes. When you visit `/wrong` , Express catches the ReferenceError and sends it to this handler instead of crashing the server.
6. `req.query` : We used `req.query` to extract the token from the URL (`?token=giveaccess`). This is different from `req.body` , which is used for POST/PUT data.